객체지향과 캡슐화에 대하여

나해수

이정하

조아진

장윤성

목차 페이지

1 객체지향과 캡슐화에 대하여

2 예시 코드

- **객체지향** : 데이터와 기능을 하나의 객체로 묶어서 객체들 간의 상호작용으로 프로그램을 동작시키는 방식

- **캡슐화** : 객체지향 프로그래밍에서 관련 있는 데이터와 데이터를 사용하는 매서드를 하나로 묶는 것

캡슐화의 핵심 개념

1. 데이터 + 매서드의 묶음

* 데이터와 데이터를 다루는 함수(메서드)를 하나의 단위로 관리한다
* 메서드는 데이터를 변경하거나 조회하는 역할을 한다

2. 데이터 보호

* 외부에서 데이터를 직접 변경하지 못하도록 한다
* 반드시 정해진 방법(매서드)을 통해서만 데이터를 사용하게 한다.

3. 접근제한자 사용

* `private` : 외부에서 직접 접근 불가 (데이터 보호 핵심)
* `public` : 메서드는 외부에서 사용할 수 있도록 공개

NEXT

객체지향

객체지향 생성 전

```java
class Student {

    // 속성(데이터)
    String name;
    int score;

    // 기능(메서드)
    void study() {
        System.out.println(name + "이 공부를 합니다.");
    }

    void showScore() {
        System.out.println(name + "의 점수는 " + score + "입니다.");
    }
}
```

객체지향 생성 후

```java
public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();

        s1.name = "철수";
        s1.score = 90;

        s1.study();
        s1.showScore();
    }
}
```

요소	의미
Student	클래스
name, score	객체의 속성
Study(), showScore()	객체의 행동
Student s1	객체 생성

캡슐화

캡슐화 사용 전

```java
class Student {

    String name;
    int score;

    void study() {
        System.out.println(name + "이 공부를 합니다.");
    }

    void showScore() {
        System.out.println(name + "의 점수는 " + score + "입니다.");
    }
}
```

private 같은 접근 제한자가 없기 때문에
외부에서 직접 접근이 가능

캡슐화 코드 적용

```java
class Student {

    private String name;
    private int score;

    public void setName(String name) {
        this.name = name;
    }

    public void setScore(int score) {
        this.score = score;
    }

    public void showScore() {
        System.out.println(name + "의 점수는 " + score + "입니다.");
    }
}
```

private 접근 제한자 사용

이로써 매서드만 사용해서 값을 설정하거나
조회 가능

END

감사합니다!